



Battle of the Elements —

A dialogue-based
game

Lucia Casiglio

Dialogue Systems — VT 2025

The game:

In this game one can fight the computer in an exciting elemental battle.

Both players choose an element amongst air, fire, water and earth, and then a character.

There are two rounds, and the moves are randomly selected for both players. Whoever wins the second round, wins the game.

In the second round, both players will boost their signature move, which could help improve their performance (or not).

```
814         | return { lastResult: event.value }
815     }
816 },
817     ASR_NOINPUT: { actions: assign({ lastResult: null }) },
818 },
819 },
820     NoInput: {
821         entry: { type: "spst.speak", params: { utterance: "I didn't catch that, please try again." } },
822         on: { SPEAK_COMPLETE: "Listen" },
823     },
824     Repair: {
825         entry: { type: "spst.speak", params: { utterance: "I heard you're unsure, what is your final choice?" } },
826         on: { SPEAK_COMPLETE: "Listen" },
827     }
828 },
829 },
830     Greeting: {
831         entry: { type: "spst.speak",
832             params: { utterance: "Welcome to battle of the elements! To exit the game, say that you want to exit,\
833                 or just say exit. To restart the game without hearing this introduction, ask to reset,\
834                 or simply say reset. If you need me to repeat a question during the game, just ask. \
835                 To play, you need to choose your element, amongst \
836                 air, fire, water and earth. Then, you will choose your player. \
837                 The game will have two rounds, and in both of them your moves will be randomly \
838                 selected by the computer. But the character you choose matters, because in the second round \
839                 they can boost specific moves, which will give you advantages or disadvantages \
840                 depending on the element you fight against. So choose wisely. Whoever wins the second\
841                 round wins the game." } },
842         on: { SPEAK_COMPLETE: "PcRandom" },
843     }
```

Platform

This project was implemented using speechstate.

Settings: same as the ones used in the labs.

```
1 import { assign, createActor, setup } from "xstate";
2 import { Settings, speechstate } from "speechstate";
3 import { createBrowserInspector } from "@statelyst/inspect";
4 import { KEY } from "C:/Users/lucia/OneDrive/Documenti/dialog system/dialogue-systems-1-2025/Code/src/azure";
5 import { DMContext, DMEvents } from "./types";
6
7 const inspector = createBrowserInspector();
8
9 const azureCredentials = {
10   | endpoint:
11   | "https://northeurope.api.cognitive.microsoft.com/sts/v1.0/issuetoken",
12   | key: KEY,
13 };
14
15 const settings: Settings = {
16   | azureCredentials: azureCredentials,
17   | azureRegion: "northeurope",
18   | asrDefaultCompleteTimeout: 0,
19   | asrDefaultNoInputTimeout: 5000,
20   | locale: "en-US",
21   | ttsDefaultVoice: "en-GB-RyanNeural",
22 };
23
```

Challenges I

Figuring out how to implement the battle was not easy, but fun.

Final decision: a specific arena for every pair of elements and different points for the same moves depending on what the two elements are.

```
25 const arenaMap: { [index: string]: {[index: string]: number} } = {
26   air_vs_water: {
27     blizzard : 4,
28     lightning : 3,
29     fog : 1,
30     whirlwind : 5,
31     frost : 2,
32     hurricane : 6,
33     "tsunami wave" : 6,
34     ice : 2,
35     "breathless bubble" : 1,
36     "rising tide" : 3,
37     "whale's swallow" : 4,
38     "whirlpool" : 5
39   },
40   fire_vs_earth: {
41     "lava flow" : 4,
42     "desert's heat" : 2,
43     "solar storm" : 5,
44     "spewing flame" : 3,
45     "burning coal" : 1,
46     "volcano storm" : 6,
47     earthquake : 6,
48     fossil : 4,
49     landslide : 5,
50     "tree's squeeze" : 2,
51     "dust storm" : 1,
52     "stone storm" : 3
53   },
```

Challenges II

The “random” factor was not easy to implement.

- Basic random function
- Function to retrieve random element and character for the computer
- Function to select the right arena based on the computer and the user's elements.
- ...

```
157 function random(n: number) {
158   let i: number = Math.floor(Math.random() * n);
159   return i;
160 }
161 function randomElemAndChar() {
162   let i: number = random(4);
163   console.log("random:", i);
164   let elem: string = elements[i];
165   let j: number = random(3);
166   let char: string = characters[elem][j];
167   console.log(`Random Choice -> Element: ${elem}, Character: ${char}`);
168   return [elem, char];
169 }
170
```

```
200 function returnArena(elem: string, pcelem: string) {
201   let possibilities: string[] = ["air_vs_water", "fire_vs_earth", "air_vs_fire", "air_vs_earth", "fire_vs_water", "water_vs_earth"];
202   let arena: string;
203   if (elem !== pcelem) {
204     arena = elem.concat("_vs_", pcelem);
205     if (!possibilities.includes(arena)) {
206       arena = pcelem.concat("_vs_", elem);
207     }
208   }
209   else {
210     let i: number = random(3);
211     const arrayWithoutElem = elements.filter(function (e) {
212       return e !== elem;
213     });
214     let other_elem: string = arrayWithoutElem[i];
215     arena = elem.concat("_vs_", other_elem);
216     if (!possibilities.includes(arena)) {
217       arena = other_elem.concat("_vs_", elem);
218     }
219   }
220   return arena;
221 }
222
```

Challenges II

- ...
- Function to retrieve a random move for both computer and user.

```
function randomMove(element: string, pcelement: string, arena: string, char: string, pcchar: string, boost: boolean = false) {  
  let i: number = random(6);  
  let j: number = random(6);  
  let move = elementsMap[element][i];  
  let pcmove = elementsMap[pcelement][j];  
  let point: number = arenaMap[arena][move];  
  let pcpoint: number = arenaMap[arena][pcmove];  
  let winner: string;  
  if (boost) {  
    if (move == booster[char]) {  
      | point = point + 1;  
    }  
    if (pcmove == booster[pcchar]) {  
      | pcpoint = pcpoint + 1;  
    }  
  }  
  
  if (point > pcpoint) {  
    | winner = char;  
  }  
  else if (pcpoint > point) {  
    | winner = pcchar;  
  }  
  else {  
    | winner = "both"  
  }  
  return [move, pcmove, winner];  
}
```

Challenges III

Considering natural interaction with the user was also a mild challenge.

- Check if the user asked to repeat the utterance.
- Check if the user changed their mind while answering.
- Check if the user asked to leave the game.
- Check if the user wants to reset the game.
- Using a function to understand more than one way of saying “yes”/”no” and for uniformity.

```
144 function repeat(utterance: string) {
145 |   return (utterance.toLowerCase().includes("repeat") || utterance.toLowerCase().includes("again") );
146 }
147 function repair(utterance: string) {
148 |   return (utterance.toLowerCase().includes("sorry") || utterance.toLowerCase().includes("wait") );
149 }
150 function exit(utterance: string) {
151 |   return (utterance.toLowerCase().includes("exit"));
152 }
153 function reset(utterance: string) {
154 |   return (utterance.toLowerCase().includes("reset"));
155 }
156
```

```
LISTEN_COMPLETE: [
  {
    target: "Exit",
    guard: ({ context }) => !!context.lastResult && exit(context.lastResult![0]?.utterance),
  },
  {
    target: "Reset",
    guard: ({ context }) => !!context.lastResult && reset(context.lastResult![0]?.utterance),
  },
  {
    target: "PcRandom",
    guard: ({ context }) => !!context.lastResult && yes(context.lastResult![0]?.utterance),
  },
  {
    target: "Greeting",
    guard: ({ context }) => !!context.lastResult && no(context.lastResult![0]?.utterance),
  },
  {
    target: ".Repair",
    guard: ({ context }) => !!context.lastResult && repair(context.lastResult![0]?.utterance),
  },
  { target: ".NoInput" },
]
```


Challenges IV

Something I had to think about when writing the battle...

- Retrieving the character chosen by the user from the object.
- Checking if the function `randomMove()` is called during the second round and boosting the character's signature move accordingly.

```
{
  target: "Battle",
  guard: ({ context }) => !!context.lastResult && characters["air"].includes(context.lastResult![0]?.utterance.toLowerCase()),
  actions: assign(({ context }) => {
    | return { char: context.lastResult![0]?.utterance.toLowerCase() }
  })),
},
```

```
112 const characters: {[index: string]: string[]} = {
113   air: ["rage", "danger", "bolt"],
114   earth: ["soul crusher", "spite", "stoneheart"],
115   fire: ["madness", "trigger", "power"],
116   water: ["bloodlust", "thirst", "nightmare"]
117 }
```

```
185 if (boost) {
186   if (move == booster[char]) {
187     | point = point + 1;
188   }
189   if (pcmove == booster[pcchar]) {
190     | pcpoint = pcpoint + 1;
191   }
192 }
```

```
129 const booster: {[index: string]: string} = {
130   rage: "frost", //fire, maybe earth
131   danger: "blizzard", //pretty much all of them
132   bolt: "lightning", //water
133   "soul crusher": "fossil", //air
134   spite: "dust storm", //water
135   stoneheart: "stone storm", //pretty much all of them
136   madness: "spewing flame", //air and earth
137   trigger: "desert's heat", //water
138   power: "burning coal", //water (very little)
139   bloodlust: "ice", //fire, maybe earth
140   thirst: "rising tide", //fire and air
141   nightmare: "whale's swallow" //water
142 }
```


Challenges V

Keeping track of the points and the battles.

- Having the app inform the user of who won and how many points they have.
- Redirect the app to the correct state according to the points and the round.
- Calculating who won at the end of the match.

```
769 | CheckPoint: {
770 |   entry: {
771 |     type: "spst.speak",
772 |     params: ({ context }) => ({
773 |       utterance: `${context.battle![2]} \
774 |       ${context.secondRound ? `goes up to ${context.battle![2]} == context.char ? `${context.mySecondpoint}` :
`${context.pcSecondpoint}`!}` : `goes up to ${context.battle![2]} == context.char ? `${context.mypoint}` : `${context.pcpoint}`!}``,
```

```
777 |   on: { SPEAK_COMPLETE: [
778 |     {
779 |       guard: ({ context }) => context.mypoint < 3 && context.pcpoint < 3,
780 |       target: "Battle"
781 |     },
782 |     {
783 |       guard: ({ context }) => (context.mypoint == 3 || context.pcpoint == 3) && !context.secondRound,
784 |       target: "SecondBattle"
785 |     },
786 |     {
787 |       guard: ({ context }) => context.mySecondpoint < 3 && context.pcSecondpoint < 3,
788 |       target: "Battle"
789 |     },
790 |     { target: "Aftermath" }
791 |   ]},
792 | },
```

```
834 |   states: {
835 |     Prompt: {
836 |       entry: {
837 |         type: "spst.speak",
838 |         params: ({ context }) => ({
839 |           utterance: `This is the end of round two, and \
840 |           ${context.mySecondpoint > context.pcSecondpoint ? `you won! ` : `the computer won! `}\
841 |           You have now reached the end of the game. To start another game right away, say reset.\
842 |           To exit the game, say exit.`
843 |         })
844 |       },
```

Challenges VI

To avoid errors when starting another game, I learnt it's always good practice to ensure resetting the variables at the end.

```
868 | Reset: {
869 |   entry: { type: "spst.speak", params: { utterance: "Resetting the game." } },
870 |   on: { SPEAK_COMPLETE: [
871 |     {
872 |       actions: assign({ pcranchoice: null, secondRound: false, mypoint: 0, pcpoint: 0, mySecondpoint: 0, pcSecondpoint: 0 }),
873 |       target: "PcRandom"
874 |     }
875 |   ]},
876 | },
877 | Exit: {
878 |   entry: { type: "spst.speak", params: { utterance: "If that's what you want, bye!" } },
879 |   on: { SPEAK_COMPLETE: [
880 |     {
881 |       actions: assign({ pcranchoice: null, secondRound: false, mypoint: 0, pcpoint: 0, mySecondpoint: 0, pcSecondpoint: 0 }),
882 |       target: "Done"
883 |     }
884 |   ]},
885 | },
886 | Done: {
887 |   on: {
888 |     CLICK: "SkipIntro",
889 |   },
890 | },
```

Key parts of the course

- Live coding
- NLU models

```
ChooseElement: {
  initial: "Prompt",
  on: {
    LISTEN_COMPLETE: [
      {
        target: "Exit",
        guard: ({ context }) => !!context.lastResult && exit(context.lastResult![0]?.utterance),
      },
      {
        target: "Reset",
        guard: ({ context }) => !!context.lastResult && reset(context.lastResult![0]?.utterance),
      },
      {
        target: "Air",
        guard: ({ context }) => !!context.lastResult && context.lastResult![0]?.utterance.toLowerCase() == "air",
        actions: assign({ elem: "air" })
      },
      {
        target: "Fire",
        guard: ({ context }) => !!context.lastResult && context.lastResult![0]?.utterance.toLowerCase() == "fire",
        actions: assign({ elem: "fire" })
      },
      {
        target: "Earth",
        guard: ({ context }) => !!context.lastResult && context.lastResult![0]?.utterance.toLowerCase() == "earth",
        actions: assign({ elem: "earth" })
      },
      {
        target: "Water",
        guard: ({ context }) => !!context.lastResult && context.lastResult![0]?.utterance.toLowerCase() == "water",
        actions: assign({ elem: "water" })
      },
    ],
  },
}
```

Development process

- Developed the battle system (arenaMap, booster ...)
- Wrote the skeleton of the app
- Wrote the random functions
- Understood how to properly assign the result of randomElemandChar()
- Figured out how to transition according to the right element
- Coded the states where the user can choose their character and figured out how to retrieve it
- Coded the battle states and created a system to keep track of the points
- Figured out how to inform the user of who won and with how many points
- Implemented the helper functions
- Fixed returnArena() for the case in which both players choose the same element
- Included the SkipIntro state to allow the user to go straight to the battle without hearing the introduction

Future developments

- NLU models to allow for a more natural dialogue
- An option to play against another user instead of the computer
- An option to choose how many battles to play
- Possibility to ask for a description of the elements' characters, their moves, and what element they are strongest against

```
const characters: {[index: string] : string[]} = {
  air: ["rage", "danger", "bolt"],
  earth: ["soul crusher", "spite", "stoneheart"],
  fire: ["madness", "trigger", "power"],
  water: ["bloodlust", "thirst", "nightmare"]
}

const elements: string[] = ["air", "fire", "water", "earth"]

const elementsMap: { [key: string]: string[] } = {
  air: ["blizzard", "lightning", "fog", "whirlwind", "frost", "hurricane"],
  fire: ["lava flow", "desert's heat", "solar storm", "spewing flame", "burning coal", "volcano storm"],
  water: ["tsunami wave", "ice", "breathless bubble", "rising tide", "whale's swallow", "whirlpool"],
  earth: ["earthquake", "fossil", "landslide", "tree's squeeze", "dust storm", "stone storm"]
};

const booster: {[index: string] : string} = {
  rage: "frost", //fire, maybe earth
  danger: "blizzard", //pretty much all of them
  bolt: "lightning", //water
  "soul crusher": "fossil", //air
  spite: "dust storm", //water
  stoneheart: "stone storm", //pretty much all of them
  madness: "spewing flame", //air and earth
  trigger: "desert's heat", //water
  power: "burning coal", //water (very little)
  bloodlust: "ice", //fire, maybe earth
  thirst: "rising tide", //fire and air
  nightmare: "whale's swallow"//water
}
```

```
Prompt: {  
  entry: { type: "spst.speak",  
    params: { utterance: "As an air warrior, you can choose one of these characters...\n    Danger, a griffin with a sharp beak and deadly claws...\n    Bolt, a lethally quick ghost made of air...\n    Rage, a giant vulture with fury in its eyes.\n    What is your choice?" } },
```

```
Prompt: {  
  entry: { type: "spst.speak",  
    params: { utterance: "As a fire warrior, you can choose one of these characters...\n    Madness, a giant lion with flames as mane...\n    Trigger, a sneaky monster, only waiting to be pushed to attack...\n    Power, a deadly dragon with eyes as burning coal.\n    What is your choice?" } },
```

```
Prompt: {  
  entry: { type: "spst.speak",  
    params: { utterance: "As a water warrior, you can choose one of these characters...\n    Bloodlust, a deadly siren with shark teeth and fins...\n    Thirst, a scary and human-sized slug, equipped to suck blood...\n    Nightmare, a mysterious creature, emerging from the darkest abyss.\n    What is your choice?" } },
```

```
Prompt: {  
  entry: { type: "spst.speak",  
    params: { utterance: "As an earth warrior, you can choose one of these characters...\n    Soul Crusher, a human-looking tree, with long fingers to squeeze its opponent...\n    Spite, an over-sized warrior-ant, eager to attack...\n    Stoneheart, a creepy gargoyle, made of indestructible stone.\n    What is your choice?" } },
```

Thank you!